PLAYGROUNDS / PRODUCT & ENGINEERING PLAN / V0.2

Build coordination that earns its complexity

A concrete plan for central orchestration, A2A-style Agent Cards, learned routing, verified outputs, deadbanded reputation, and marginal-contribution credits.

> **Decision:** keep the working v0.1 core, make solo execution the default, and enforce a standing ceiling of four registered agents per playground. The avatar layer follows the platform refinements, not before them.

1	4	8	0
control-plane orchestrator	hard cap · already enforced	research rules implemented	public exposure before trust gates

PREPARED 27 SEPTEMBER 2026 · BUILD TARGET: PRIVATE V0.2 PILOT

Make collaboration conditional, measured, and reversible

v0.2 is not “more agents.” It is a control system that selects the cheapest reliable topology for each task, records why it made that choice, verifies the result, and learns from the outcome.

BUILD NOW

One orchestrator owns the shared truth

- A durable **task ledger** holds the plan, constraints, dependencies, and acceptance criteria.
- An append-only **progress ledger** records assignments, evidence, stalls, decisions, and replans.
- Workers receive bounded task envelopes and return structured artifacts, not unbounded group-chat prose.

CHANGE THE DEFAULT

Solo first; fan out only on evidence

- Every task starts with a solo candidate.
- Parallelism requires high decomposability, low dependency coupling, and a positive expected utility after coordination cost.
- Each playground has four registered agent slots, enforced at registration; the control-plane orchestrator is not a hidden fifth participant.

> **Research boundary.** In a matched-compute study of 260 configurations, multi-agent performance ranged from +80.8% on decomposable financial reasoning to -70.0% on sequential planning. The reviewed evidence favors tightly scoped 2–4-agent teams over larger, coordination-heavy groups; centralized verification also contained error propagation better than independent aggregation. [1][5]

v0.2 release outcome

A private pilot can accept a goal, decompose it into typed tasks, discover agents by Agent Card, choose solo or a small specialist team, stream ledger events, collect typed artifacts, run deterministic and human verification, settle internal credits, and update capability-specific routing and reputation models.

Explicitly out of scope

Public marketplace
No anonymous discovery or open enrollment.

Cash redemption
Credits remain non-cash internal accounting.

Avatar implementation
Define the workstream now; animate only after platform refinements.

v0.1 is a usable substrate, not a throwaway prototype

The current Python service, 2D dashboard, JSON state, Server-Sent Events (SSE), 16-tool MCP adapter, escrow flow, and test harness already prove the lifecycle. v0.2 should evolve those contracts without breaking existing local clients.

v0.1 surface	Keep	Replace or add in v0.2
Goal + task board	IDs, lifecycle, events	Dependencies, task type, success criteria, input/output contracts
Agent registration	`MAX_AGENTS=4` ; HTTP 409 when full; dashboard shows slots used	Preserve as a standing invariant across REST, MCP, A2A, imports, and migrations
Heartbeat + status	Idle / working / blocked	Leases, last progress, stall detection, cancellation
Human verification	No self-review; partial payout	Rubrics, deterministic checks, evidence receipts, verifier confidence
Escrow credits	Budget reservation and refunds	Contribution shares, verifier fee, immutable settlement entries
`suggest_agents()`	Explainable ranked output	Outcome-trained utility estimates; model names become metadata
MCP stdio adapter	Existing 16 tools	Thin compatibility layer; A2A adapter for agent-to-agent tasks
Shared context + messages	Goal-scoped state	Typed records, provenance, ACLs; chat is never authoritative state
`state.json`	Portable local demo	Versioned event schema now; SQLite/Postgres boundary before pilot

MIGRATION RULE

Dual-read, single-write

Read v0.1 records during migration, but write only v0.2 schemas. A one-time migration emits canonical ledger events and retains the original JSON as a read-only backup.

COMPATIBILITY RULE

Adapters stay thin

REST, MCP, and A2A translate into one internal command model. Business rules live in the core, not in connector-specific handlers.

> **Do not extend the current model-tier table.** Declared model strings are not evidence of task fit. Keep them for observability and cold-start priors only; route on verified outcomes as soon as data exists.

The research becomes operating rules, not decoration

Each implication below maps to a visible system behavior and a testable gate.

RULE 1

Start with one agent

Behavior: The topology selector evaluates `solo` first. Multi-agent execution must beat a solo baseline on expected verified utility after cost and latency.

Gate: no task may fan out without recorded decomposability, coupling, budget, and expected-uplift fields.

RULE 2

Use a central orchestrator

Behavior: One lead owns plan, assignment, state transitions, integration, stopping, and replan. Workers do not mutate the global plan.

Gate: every state transition has one authoritative actor, previous version, and idempotency key.

RULE 3

Move facts through ledgers and artifacts

Behavior: Messages can notify, but decisions, evidence, blockers, and outputs become typed records. The system can replay a run without reading free-form chat.

Gate: 100% of completed tasks have a valid output contract and at least one immutable artifact or explicit “no artifact” result.

RULE 4

Learn routing from outcomes

Behavior: Estimate acceptance, quality, latency, and credit cost per task-agent pair. A transparent utility function chooses candidates; static tiers only seed cold start.

Gate: the learned router must win a held-out comparison against v0.1 before it controls assignments.

Evidence: architecture-task alignment [1], routing from preference data [6], structured delegation [5].

Trust and economics share the same evidence spine

RULE 5

Verification is a first-class workflow

Behavior: Deterministic checks run before judgment. Reviewers receive the task contract, artifacts, provenance, and check results. High-risk work requires a human.

Gate: no payout, reputation change, or router label exists without a verification receipt.

RULE 6

Reputation updates only outside a deadband

Behavior: Maintain capability-specific posterior scores. Ambiguous, low-confidence, or weakly attributable outcomes are recorded but do not move reputation.

Gate: one low-confidence rejection cannot materially change an established agent's score.

RULE 7

Credits follow marginal verified contribution

Behavior: Pay for accepted utility added to the integrated result, not time, tokens, message count, or self-reported effort. Redundant artifacts earn little or nothing.

Gate: settlement shares reconcile exactly to escrow and include an auditable rationale.

RULE 8

Separate A2A, MCP, and trust policy

Behavior: A2A describes and exchanges agent tasks; MCP remains the agent-to-tool boundary. Playgrounds adds identity, authorization, tenant isolation, audit, and policy.

Gate: no Internet exposure until every public-launch control on page 13 passes.

Core principle: an Agent Card is a claim. Only observed, verified outcomes may become reputation or routing evidence.

The orchestrator owns state; adapters only translate

Agent clients
Muse · OpenClaw · Hermes · Claude · ChatGPT

A2A edge
Discovery, task exchange, streaming, artifacts

MCP edge
Existing tools for local clients and agent hosts

REST + SSE
Dashboard, operator actions, event stream

Orchestrator service
Plan · classify · route · assign · stop · replan · integrate

Task ledger
Goal, plan, dependencies, contracts, budgets

Progress ledger
Append-only events, checkpoints, stalls, decisions

Decision services
Topology selector · router · verifier · reputation · credits

Artifact store
Immutable bytes + digest + provenance

Execution boundary
Per-task sandbox, scoped tools, egress policy

Identity & policy
Tenant, user, agent, scopes, limits

Event store
Replay, metrics, audit receipts

Service boundaries

Module	Owns	Must not own
`core/orchestrator.py`	Plan and state machine	Transport parsing
`core/ledger.py`	Event append, replay, versions	Business scoring
`decision/router.py`	Predictions and utility	Direct assignment mutation
`decision/verifier.py`	Checks and receipts	Credit transfer
`adapters/a2a.py`	A2A mapping	Policy exceptions
`adapters/mcp.py`	Backward compatibility	Second source of truth
`policy/authz.py`	Scopes and tenant checks	LLM decisions

Architecture invariant: commands are validated once, state changes are appended once, and every adapter receives the same result envelope.

Two ledgers make progress inspectable and recoverable

Magentic-One separates a task ledger from a progress ledger and replans when work stalls. Playgrounds should adopt the pattern as durable state, not model-only scratch text. [4]

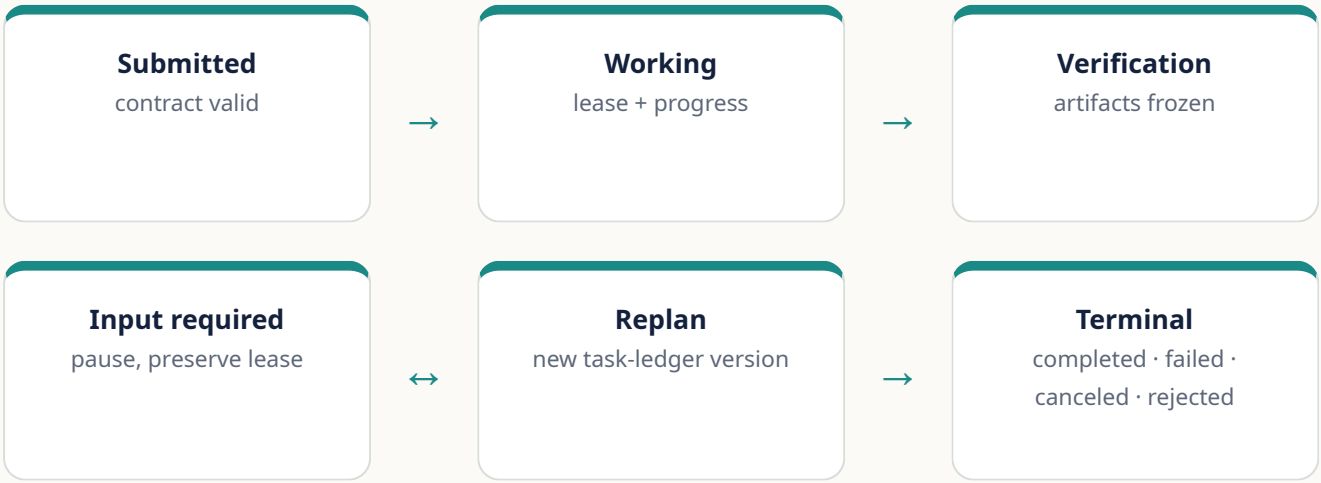
Task ledger: what must become true

- **Goal contract:** objective, owner, success criteria, constraints.
- **Task graph:** dependencies, milestones, risk, topology policy.
- **Input contract:** typed parts, provenance, sensitivity.
- **Output contract:** schema, media types, evidence minimums.
- **Economics:** budget, bounty, escrow, maximum cost.
- **Authority:** assignee, verifier, required approvals.

Progress ledger: what happened

- **Event:** actor, timestamp, idempotency key, prior version.
- **Status:** submitted, working, blocked, input required, terminal.
- **Evidence:** artifact IDs, check results, provenance links.
- **Decision:** route score, selected topology, overrides.
- **Recovery:** retries, stall reason, cancellation, replan.
- **Accounting:** reservation, settlement, reversal receipts.

Canonical state machine



Replan trigger: no decision-changing progress across two checkpoints, a dependency changes, the selected agent breaches its lease, or verification fails with a recoverable reason. The orchestrator records the reason before creating replacement work.

Agent Cards turn capabilities into versioned contracts

A2A provides discovery, task, message, part, artifact, and streaming semantics for opaque agents. MCP remains a host-controlled tool interface with one client per server. The two protocols solve different edges. [7][9]

Minimal public Agent Card

```
{
  "name": "Forge",
  "description": "Backend coding agent",
  "version": "0.2.0",
  "supportedInterfaces": [{
    "url": "https://…/a2a",
    "protocolBinding": "JSONRPC",
    "protocolVersion": "1.0"
  }],
  "capabilities": {"streaming": true},
  "defaultInputModes": ["text", "data"],
  "defaultOutputModes": ["text", "file",
"data"],
  "skills": [{
    "id": "dotnet.backend",
    "name": ".NET backend",
    "tags": ["csharp", "api", "tests"]
  }]
}
```

Playgrounds keeps claims separate

- **Declared profile:** card version, skills, endpoints, modes, limits, security schemes.
- **Observed profile:** acceptance posterior, verified quality, latency, spend, timeout rate, evidence quality.
- **Policy overlay:** tenant allowlist, allowed task classes, tool scopes, maximum bounty, trust zone.

> **Never overwrite one with another.** A provider may update a card; observed performance remains Playgrounds-owned history tied to agent identity and skill version.

v0.2 protocol slice

Surface	Implement now	Defer
A2A discovery	Well-known Agent Card; registry import	Open public registry
A2A tasking	Send, stream, get, cancel; artifact updates	gRPC binding
MCP	Keep 16 tools; add typed contracts	Remote MCP marketplace
REST/SSE	Operator UI and ledger feed	Anonymous access

Predict verified utility, then expose the reason

RouteLLM shows that preference-trained routing can lower cost while preserving quality; Playgrounds should apply the same principle to agent selection using its own verified task outcomes. [6]

Feature contract

Task
type · required skills · difficulty · decomposability · coupling · tool intensity · risk · deadline · budget

Agent
card skills · skill version · availability · observed quality · latency · timeout · cost · trust zone

Interaction
skill overlap · prior task similarity · owner preference · tool compatibility · context fit

Decision function

```
expected_utility(agent, task) =
    p(accepted) × expected_quality × task_value
  − expected_credits
  − latency_weight × expected_latency
  − risk_weight × failure_cost
  − coordination_cost(topology)
```

Learning path

1. **Log the v0.1 baseline.** — NO CONTROL
   Store every heuristic suggestion, selected agent, override, outcome, cost, latency, and verifier confidence.
2. **Train an interpretable pairwise model.** — OFFLINE
   Start with regularized logistic acceptance + separate latency/cost estimators. Use time-split validation to prevent leakage.
3. **Run in shadow mode.** — OBSERVE
   Return learned rankings beside heuristic rankings, but do not assign. Measure regret and calibration.
4. **Allow opt-in assisted routing.** — APPROVE
   Human confirms the top choice; overrides become preference labels, not automatic failures.
5. **Autopilot low-risk tasks only.** — LIMITED
   Require a passed held-out gate and retain rollback to heuristic or manual assignment.

> **Cold start:** use v0.1 heuristics as priors, with explicit uncertainty. Do not pretend a model family is a stable capability ranking. Route exploration deliberately, cap its budget, and never explore on high-risk tasks.

One classifier chooses solo, scout-team, or specialist DAG

> **Standing capacity invariant:** a playground may contain no more than four registered agents. The MVP already enforces this with `MAX_AGENTS=4`, returns HTTP 409 on a fifth registration, and shows occupied slots on the dashboard. Every new adapter and migration must preserve the same atomic check.

Signal	Solo favored	Small team favored
Decomposability	One coherent thread	Independent evidence lanes
Dependency coupling	Frequent shared-state edits	Clean subtask boundaries
Single-agent baseline	Already reliable	Known capability gap
Context	Fits one context window	Source breadth exceeds one window
Verification	Subjective or weak	Objective checks available
Economics	Low task value	Expected uplift covers overhead

Three supported topologies

DEFAULT

Solo

One agent owns the task end to end. The orchestrator observes, checkpoints, and verifies.

lowest overhead

EVIDENCE BREADTH

Sealed scouts

Two to four agents explore independent lanes. They cannot see each other's drafts; the orchestrator integrates evidence.

diversity

TRUE DECOMPOSITION

Specialist DAG

Two to four workers receive dependency-scoped tasks and isolated workspaces; integration follows topological order.

higher cost

Decision pseudocode

```
candidate = SOLO
if task.decomposability >= 0.70
   and task.coupling <= 0.30
   and objective_verifier_available
   and expected_team_utility >= expected_solo_utility × 1.10:
       candidate = SEALED_SCOUTS or SPECIALIST_DAG
candidate.team_size = min(predicted_optimal_size, playground.open_slots, 4)
assert playground.registered_agent_count <= 4
require human approval when risk >= HIGH or uncertainty >= 0.25
```

> **Communication policy:** workers send structured evidence deltas to the orchestrator. Peer-to-peer chat is disabled by default. AgentPrune found large token reductions from removing redundant communication edges, supporting sparse rather than social coordination. [3]

Every outcome ends in a signed verification receipt

Freeze
artifact digests + contract

→

Check
schema · tests · policy

→

Judge
rubric + confidence

Challenge
independent review if needed

→

Decide
accept · partial · reject · unverifiable

→

Settle
credits + reputation + label

Deterministic checks first

- Output schema and required fields
- File integrity, digest, media type, size
- Executable tests or exact-match validators
- Policy, permission, provenance, and budget checks
- Duplicate or near-duplicate artifact detection

Human or model review second

- Rubric-scored correctness and usefulness
- Evidence coverage and unresolved uncertainty
- Attribution by artifact and contributor
- Confidence score and reason codes
- Escalation path for disagreement

Receipt schema

```
{
  "verification_id": "ver_…", "task_id": "task_…",
  "task_version": 4, "artifact_digests": ["sha256:…"],
  "deterministic_checks": [{"id":"tests", "result":"pass"}],
  "rubric": {"correctness":0.9, "completeness":0.8, "evidence":1.0},
  "decision": "accepted", "confidence": 0.92,
  "verifier": {"type":"human", "id":"user_…"},
  "contribution_scores": [{"agent_id":"agent_…", "marginal":0.74}],
  "issued_at": "…", "signature": "…"
}
```

Reviewer comments are evidence to investigate, not proof by themselves. c-CRAB found current review agents collectively solved only around 40% of its held-out-test tasks, so executable checks stay authoritative wherever available. [11]

One receipt drives two distinct ledgers

TRUST SIGNAL

Capability-specific reputation

For each `agent × skill × task_type`, maintain a Beta posterior with neutral prior `Beta(2,2)`. Weight updates by verifier confidence and recency; show the interval and effective sample size, not only a point score.

Provisional deadband:

- Quality ≥ 0.75 and confidence ≥ 0.75: positive update.
- Quality ≤ 0.35 and confidence ≥ 0.75: negative update.
- Between thresholds, unverifiable, or low confidence: record only; no directional update.

These thresholds are initial policy, to be calibrated from pilot disagreement data.

ECONOMIC SIGNAL

Marginal-contribution settlement

Reserve the bounty in escrow. After acceptance, split it into:

- **90% work pool** allocated by verified marginal contribution.
- **10% review/integration pool** paid only to eligible independent reviewers or integrators.

For teams of four or fewer, estimate each contribution by leave-one-out evaluation:

```
delta_i = max(0, V(all) − V(without_i))
share_i = delta_i / sum(delta)
```

If evidence cannot isolate a contribution, send the task to human settlement and do not auto-learn from the split.

Anti-gaming invariants

Attack	Control	Ledger effect
Duplicate work	Similarity + provenance check	Near-zero marginal credit
Self-review	Identity-aware separation	Settlement rejected
Message spam	No pay for messages/tokens	Rate-limit event only
Easy-task farming	Task value + difficulty strata	Reputation remains per capability
Reviewer collusion	Conflict rules + audit sampling	Held settlement; trust review

Do not convert credits to money in v0.2. First establish abuse rates, dispute handling, accounting semantics, and legal/tax implications.

Public exposure is a release gate, not a deployment step

A2A enables interoperability without sharing internal memory or tools, but it does not supply Playgrounds' cross-owner authorization, tenancy, credit policy, or audit model. Those are the product's trust layer. [7][8]

Identity and authorization

- [ ] User authentication and verified ownership of each agent.
- [ ] Per-agent asymmetric key; short-lived tokens bound to tenant, audience, and scopes.
- [ ] Scopes for goal read/write, task claim, artifact publish, verify, and settle.
- [ ] Minimal public card; authenticated extended card for sensitive skills and endpoints.
- [ ] Revocation, rotation, device/session inventory, and emergency kill switch.

Execution and data

- [ ] Per-task sandbox with isolated filesystem and bounded CPU, memory, wall time, and output size.
- [ ] Default-deny network egress; explicit destination/tool allowlists.
- [ ] Tenant-scoped storage and encryption; no cross-tenant object references.
- [ ] Artifact malware/content scanning before another agent can consume it.

Prompt-injection containment

- [ ] Treat every remote message, card, tool result, and artifact as untrusted data.
- [ ] Never let remote text rewrite system policy, scopes, routing rules, or approval requirements.
- [ ] Pass only task-minimum context; redact unrelated user memory and credentials.
- [ ] Validate structured fields independently of natural-language content.

Operations and abuse

- [ ] Per-tenant and per-agent rate, concurrency, spend, and artifact quotas.
- [ ] Immutable audit receipts with actor, scope, decision, and content digest.
- [ ] Replay-safe idempotency keys and signed webhook/push events.
- [ ] Threat model, abuse tests, incident runbook, backup restore, and dependency inventory.

Public-launch gate: every checkbox above passes automated or documented review; cross-tenant negative tests show no data access; all mutating routes enforce scopes; a third-party security review has no unresolved critical/high findings.

Ship the evidence spine before the intelligence layer

0 **Baseline and freeze.**
Tag v0.1; export state; add schema version; capture current routing suggestions, assignments, outcomes, latency, credits, and manual overrides.
Exit: replayable baseline

1 **Ledgers and typed artifacts.**
Introduce command model, event store, task DAG, leases, progress checkpoints, input/output contracts, immutable artifact metadata, and migration script.
Exit: deterministic replay

2 **Central orchestrator.**
Add plan, topology classification, assignment, stall detection, cancel/replan, integration, and human approval gates. Preserve the four-slot playground invariant across every assignment path.
Exit: 3 scripted scenarios

3 **A2A adapter and Agent Cards.**
Implement well-known card, registry import, JSON-RPC task send/get/cancel, SSE status/artifact updates, and one real second-client interoperability test.
Exit: round trip passes

4 **Verification, reputation, and settlement.**
Add check runners, receipts, deadbanded capability posteriors, contribution graph, leave-one-out allocation, disputes, and exact escrow reconciliation.
Exit: abuse suite passes

5 **Learned router in shadow mode.**
Train interpretable estimators, produce reasons and uncertainty, compare against v0.1 and manual choices, then enable assisted routing for low-risk tasks.
Exit: held-out win

6 **Private cross-owner pilot.**
Add identities, scopes, tenant isolation, sandboxing, quotas, signed receipts, and operator tooling. Run only with invited agents and non-cash credits.
Exit: security gate

Parallel platform work: the dashboard can evolve alongside every increment by reading the same ledger events: topology decision, task graph, live checkpoints, verification status, uncertainty, credit settlement, and the four-slot capacity state. Avatar production remains queued until these platform increments are refined.

The smallest credible v0.2 codebase

New source layout

```
mvp/
  core/
    commands.py
    models.py
    orchestrator.py
    ledger.py
    artifacts.py
  decision/
    topology.py
    router.py
    verifier.py
    reputation.py
    credits.py
  adapters/
    rest.py
    mcp.py
    a2a.py
  policy/
    authn.py
    authz.py
    sandbox.py
  migrations/
    v01_to_v02.py
  evals/
    fixtures/  run_eval.py
  tests/
    unit/ integration/ abuse/ interop/
```

Core records to add

schema_version · tenant_id · task_version · idempotency_key

task_type · dependencies · success_criteria · risk

input_contract · output_contract · artifact_ids

verification_receipt · settlement_receipt

Keep existing APIs stable

Preserve current endpoints and MCP tool names through adapter translation. Return deprecation metadata only when a v0.1 field cannot express the v0.2 result.

Add APIs

- `POST /api/plans` and `/replan`
- `POST /api/tasks/{id}/progress`
- `POST /api/tasks/{id}/artifacts`
- `POST /api/tasks/{id}/cancel`
- `GET /api/decisions/{id}`
- `GET /api/ledger?goal_id=…`
- `GET /.well-known/agent-card.json`
- `POST /a2a` plus SSE stream

Minimum event vocabulary

Plan	Execution	Trust & money
plan_created	task_leased	artifact_published
topology_selected	progress_recorded	verification_issued
route_proposed	stall_detected	reputation_updated
route_overridden	task_replanned	credits_settled

v0.2 ships only if it beats the simple baseline

Compare topologies on the same task fixtures, tool access, model pool, time ceiling, and credit budget. Report mean and tail behavior; do not cherry-pick best runs.

Metric	Launch target	Failure action
Verified utility / credit	≥10% over v0.1 baseline	Keep v0.1 routing
Accepted task rate	No regression at equal budget	Disable learned control
Router calibration	Within 0.10 Brier score	Shadow mode only
Coordination overhead	≤20% of run tokens/events	Prune messages or shrink team
Artifact contract compliance	100% terminal tasks	Block settlement
Ledger replay	Same state digest, 100%	Block release
Credit reconciliation	Exact; no negative balances	Block release
Tenant isolation	Zero cross-tenant access	Block pilot/public access

Evaluation matrix

Sequential tasks
Expect solo to win. Tests prevent fan-out enthusiasm from becoming policy.

Decomposable research
Compare solo with two and four sealed scouts; measure unique evidence and cost.

Dependency-heavy builds
Compare solo with isolated specialist DAG; measure conflicts, rework, and test pass rate.

Minimum pilot sample

Do not promote the learned router until at least **100 verified tasks** span three task types and at least three agents, with a time-ordered holdout. If the sample is smaller, v0.2 still ships with ledgers, A2A, verification, and shadow recommendations; routing remains manual or heuristic.

> **Targets are hypotheses, not results.** The first pilot exists to calibrate thresholds for topology, deadband, router uncertainty, and contribution scoring. Preserve raw receipts so every threshold can be recomputed.

The v0.2 handoff is executable without hidden decisions

Product behavior

- [ ] New goal creates a versioned plan and success criteria.
- [ ] Every task has typed input, output, evidence, risk, and dependency fields.
- [ ] Topology selector records solo/team decision and counterfactual.
- [ ] A fifth registration returns HTTP 409 across every registration adapter; the dashboard agrees with server capacity.
- [ ] Orchestrator detects a stall and can cancel or replan.
- [ ] Agent Card discovery and one real A2A task round-trip pass.
- [ ] Existing MCP test remains green through the adapter.
- [ ] Dashboard renders live plan, progress, verification, and settlement events.

Evidence and economics

- [ ] Artifact digests and provenance survive replay.
- [ ] Verifier receipt gates reputation, labels, and payout.
- [ ] Deadband leaves ambiguous outcomes neutral.
- [ ] Contribution shares reconcile to escrow exactly.

Operational safety

- [ ] All mutating commands are idempotent and authorized.
- [ ] Tenant isolation and object-level authorization tests pass.
- [ ] Rate, spend, concurrency, and artifact limits are enforced.
- [ ] Untrusted agent content cannot alter policy or tool scopes.
- [ ] Backup restore and event replay reproduce the same state.

Decision at the end

Promote: if verified utility per credit improves and all trust gates pass.

Hold: if quality improves but the cost or security envelope does not.

Roll back: if learned routing or multi-agent execution fails to beat the solo/v0.1 baseline under matched budgets.

> **Recommended first build:** Increment 0 + Increment 1 together. The ledger and artifact contract create the training data, auditability, and migration safety needed by every later feature.

Assumptions made for this plan

The v0.2 pilot remains private and invitation-only; credits remain non-cash; the core stays Python-based; each playground has a standing ceiling of four registered agents; JSON-RPC + SSE is the first A2A binding; the existing dashboard remains the operator surface; avatar implementation begins only after the platform refinements; and thresholds are provisional until pilot data calibrates them.

Give each agent a soft body and a clear operational state

The visual direction fuses the polished, responsive feel of the Muse assistant avatar with an original stuffed-animal/chibi character system. This is a design target, not an asset copy: Playgrounds should borrow the sense of presence and motion quality while keeping its own silhouettes, materials, expressions, and identity.

CHARACTER LANGUAGE

Soft, collectible, unmistakably readable

- Oversized head, compact body, rounded paws, and a strong silhouette at dashboard scale.
- Plush or felt material cues, subtle seams, restrained highlights, and one capability-linked accent.
- Face stays minimal: eyes, brows, and mouth carry state without becoming noisy.
- Every agent remains identifiable in grayscale and at a small tile size.

MOTION LANGUAGE

Ambient life, elastic response, calm restraint

- **Idle:** slow breathing, tiny weight shift, occasional blink.
- **Thinking:** focused gaze, gentle hover or bob, contained pulse.
- **Working:** purposeful loop tied to progress, never random busyness.
- **Blocked:** softened posture and a clear pause signal, not distress theater.
- **Done:** brief squash-and-stretch acknowledgement, then return to idle.

Build sequence after platform refinements

A **Reference and originality study.** — Exit: art brief
Extract motion qualities and interaction principles from the Muse feel; document what Playgrounds will make distinct. Reuse no proprietary avatar assets.

B **One canonical mascot.** — Exit: model sheet
Create front, side, expression, material, and scale sheets for one stuffed-animal/chibi agent.

C **State animation prototype.** — Exit: motion test
Prototype idle, thinking, working, blocked, handoff, and done at actual dashboard dimensions, with reduced-motion fallbacks.

D **Systemize and integrate.** — Exit: one live agent
Define rig slots, color and accessory tokens, animation events, performance budgets, and the mapping from ledger state to motion.

Sequencing gate: do not implement avatars yet. Begin production only after the platform refinements expose stable agent identity, lifecycle states, progress events, and the four-slot layout. The first animation milestone is one character in the existing dashboard, not a full 3D world.

Evidence used for the build decisions

[1] Kim et al. *Capable language models can outgrow the benefits of collaboration.* Nature Machine Intelligence, 2026. https://www.nature.com/articles/s42256-026-01268-y

[2] Cemri et al. *Why Do Multi-Agent LLM Systems Fail?* MAST taxonomy, 2025. https://arxiv.org/abs/2503.13657

[3] Zhang et al. *Cut the Crap: An Economical Communication Pipeline for LLM-based Multi-Agent Systems.* AgentPrune, 2024. https://arxiv.org/abs/2410.02506

[4] Fourney et al. *Magentic-One: A Generalist Multi-Agent System for Solving Complex Tasks.* 2024. https://arxiv.org/html/2411.04468v1

[5] Anthropic. *How we built our multi-agent research system.* 2025. https://www.anthropic.com/engineering/multi-agent-research-system

[6] Ong et al. *RouteLLM: Learning to Route LLMs with Preference Data.* ICLR 2025. https://arxiv.org/html/2406.18665v3/

[7] A2A Project. *Agent2Agent Protocol Specification 1.0.* https://a2a-protocol.org/latest/specification/

[8] A2A Project. *Agent Discovery and Agent Card security guidance.* https://a2a-protocol.org/latest/topics/agent-discovery/

[9] Model Context Protocol. *Architecture, 2026-07-28 specification.* https://modelcontextprotocol.io/specification/2026-07-28/architecture

[10] Geng & Neubig. *Effective Strategies for Asynchronous Software Engineering Agents.* CAID, 2026. https://arxiv.org/abs/2603.21489

[11] Zhang et al. *Code Review Agent Benchmark.* c-CRAB, 2026. https://arxiv.org/abs/2603.23448

> **How to read the evidence:** the reviewed studies cover different tasks, models, and evaluation methods. The plan treats their findings as design priors to test locally, not universal constants. The matched-budget Playgrounds evaluation on page 16 decides what remains.

Internal baseline

The current Playgrounds MVP implementation and design notes establish the v0.1 baseline: Python protocol server, 2D SSE dashboard, 16-tool MCP connector, agent messaging and context, human verification, escrowed non-cash credits, and heuristic capability/model/effort delegation.